PROJECT PROPOSAL

SDF04 – Object-Oriented Programming

Project Title:

MediVault: A Secure Medical Information Management and Emergency Access Control System

Subject Code	SDF04
Subject Title	Object-Oriented Programming (Python & Database)
Application Type	Web-Based GUI Application (Non-Console)
Prepared By	Obrero, Jabyn Eriko G Paje, Vien Destiny Mavhic R. Sanchez, Ronald Dean M.
Course / Year / Section	LFCA311E064
Instructor	Ms. Jessa Evangelista
Date Submitted	09 /19/ 2026

1. Introduction / Background

Healthcare providers routinely need fast, accurate access to a patient's medical history, allergies, and current medications, yet many small clinics and care settings still rely on paper charts or disconnected digital files that are slow to search and easy to misplace. This becomes especially dangerous in emergencies, when a patient may be unable to communicate their own medical history and responders have only minutes to identify life-threatening allergies or drug interactions. At the same time, medical records are among the most sensitive categories of personal data, so any digital system that centralizes them must also enforce strict, role-based access so that only authorized personnel can view or modify a given patient's information.

This tension between fast access during emergencies and strict privacy during routine operations is the problem MediVault is designed to address. The system needs to model distinct entities such as patients, medical records, staff roles, permissions, and access events, and manage the relationships between them (a patient has many medical records, a staff role determines which fields can be viewed, an emergency access event is logged against a specific patient and user), which makes it well suited to an object-oriented design built in Python with a relational database backend. Object-Oriented Programming principles allow these real-world entities and their permission boundaries to be modeled directly as classes, while the database ensures records and access logs persist reliably over time.

A software solution is therefore an appropriate response: it can enforce role-based access control consistently, provide a controlled emergency-access pathway that still keeps every view accountable through logging, and present the information through an interface simple enough for healthcare staff to use quickly under pressure.

2. Project Description

MediVault is a secure web-based application for managing patient medical information while ensuring that sensitive records are accessible only to authorized healthcare personnel. It lets healthcare users manage patient profiles, medical histories, allergies, medications, and emergency medical information, with each user's access

shaped by their assigned role. It also includes a controlled emergency-access feature that lets authorized personnel view essential medical details during urgent situations, while every access, routine or emergency, is logged to preserve privacy and accountability. The primary users are healthcare personnel such as physicians, nurses, records staff, and emergency responders, along with administrators who oversee access policies and audit logs. Rather than a console-based program requiring typed commands, MediVault is built as a Graphical User Interface (GUI) delivered through a web browser, so that patient records and emergency information can be reviewed and updated through forms, dashboards, and clearly labeled screens.

3. Objectives

General Objective:

To design and implement a web-based GUI application that securely manages patient medical information through role-based access control, while providing a controlled, accountable emergency-access pathway for authorized personnel.

Specific Objectives:

1. Design and implement a graphical user interface for managing patient profiles, medical histories, allergies, and medications.

2. Apply Object-Oriented Programming principles (encapsulation, inheritance, polymorphism, abstraction) in the design of the records, access-control, and emergency-access subsystems.

3. Design and integrate a relational database to store and manage patients, medical records, staff roles, permissions, and access logs.

4. Implement core CRUD (Create, Read, Update, Delete) functionality for patient records through the GUI, scoped by each user's role.

5. Ensure data validation, error handling, and basic security (e.g., login authentication, password hashing, and encryption of sensitive fields) within the application.

6. Implement a role-based access control system that restricts which fields and actions each user role can view or edit.

7. Provide a controlled emergency-access feature that surfaces essential medical information quickly while automatically logging the access event for accountability.

4. Scope and Limitations

Scope:

- The application will be a GUI-based web program developed using Python (Flask) with an HTML/CSS/JavaScript front end.
- It will include the following core modules: Login / Role-Based Authentication, Patient Profile Management, Medical History Management, Allergy & Medication Tracking, Emergency Access Module, Access Logging & Accountability, and an Admin Dashboard.
- Data will be stored and retrieved using a MySQL database.
- Sensitive fields (e.g., medical history, medications) will be protected through role-based permissions and, where appropriate, field-level encryption.

Limitations:

- The system will not include a native mobile application, cloud deployment, or integration with external hospital information systems unless stated otherwise by the instructor.

- The emergency-access feature grants authorized personnel expedited viewing of essential records within the system; it does not interface with real ambulance dispatch or hospital emergency systems.
- The system does not provide medical diagnosis, treatment recommendations, or clinical decision support beyond presenting the stored records.

5. System Features / Modules

Module / Feature	Description	Key OOP Class(es) Involved
Login / Role-Based Authentication	Allows healthcare personnel to securely log in, with access rights determined by their assigned role (e.g., physician, nurse, records staff, emergency responder).	User, Role, AuthManager
Patient Profile Management	Enables authorized staff to create, view, update, and manage patient demographic and identification information.	Patient, PatientProfile
Medical History Management	Records and maintains a patient's medical history, past diagnoses, and treatment notes.	MedicalRecord, HistoryEntry
Allergy & Medication Tracking	Stores and updates a patient's known allergies and current or past medications.	Allergy, Medication
Role-Based Access Control (RBAC)	Restricts which fields and actions each user role can view or edit, enforcing least-privilege access to sensitive records.	AccessControl, Permission
Emergency Access Module	Provides authorized personnel a controlled way to view essential medical details (allergies, medications, emergency contacts) during urgent situations, even outside normal access rules.	EmergencyAccess, AccessRequest
Access Logging & Accountability	Automatically records who accessed which patient's record, when, and under what circumstance (routine or emergency), to support privacy audits.	AccessLog, AuditTrail
Admin Dashboard & Reporting	Gives administrators an overview of system usage, emergency-access events, and audit reports.	Dashboard, ReportGenerator

6. Tools and Technologies

Programming Language	Python
GUI / Web Framework	Flask (backend) with HTML5, CSS3, and JavaScript for the web GUI
Database	MySQL
Database Connector Library	mysql-connector-python / SQLAlchemy

Security Libraries	bcrypt (password hashing), Flask-Login (session/role management), cryptography (data encryption for sensitive fields)
IDE / Editor	VS Code
Version Control	Git and GitHub
Other Libraries	Flask-WTF (form validation), reportlab (PDF export of records/audit logs)

7. Project Timeline (Example Estimated)

Phase	Activities	Target Date
Planning & Design	Requirements gathering, role/permission matrix, ERD, class diagram, and UI mockups for patient records and emergency access.	Week 1-2
Database Setup	Design and create schema for patients, records, allergies, medications, roles, and access logs; load sample/test data.	Week 3
GUI Development	Build patient profile forms, medical history views, RBAC-restricted screens, and the emergency-access interface.	Week 4-5
Backend / OOP Logic	Implement core classes, role-based access control, emergency-access workflow, and database integration.	Week 6-7
Testing & Debugging	Unit testing of access-control rules and emergency-access logic, input validation checks, and bug fixing.	Week 8
Documentation & Presentation	Finalize project report, prepare demo and final presentation.	Week 9